

Topic 3: Introduction to Databases

This topic introduces you to the world of relational databases. You'll learn the language used to talk to them, **SQL**, and the fundamental operations you can perform. We'll also cover how to design a robust database schema using **Entity-Relationship (ER) Diagrams** and enforce data integrity with **constraints**.

Objectives and Key Results

Objective	Key Result
Grasp the fundamentals of SQL and relational databases.	You can explain what a relational database is, create a new database and tables in PostgreSQL, and define appropriate data types for table columns.
Master basic database operations.	You can confidently write INSERT, SELECT, UPDATE, and DELETE queries to perform CRUD operations on a PostgreSQL database.
Understand and enforce data integrity.	You can define and explain the purpose of PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE constraints, and apply them to tables in PostgreSQL.
Model data requirements effectively.	You can design a logical database structure by creating a clear and well-defined ER diagram that includes entities, attributes, and relationships.

3.1 What is SQL?

Structured Query Language (SQL) is the standard programming language for managing and manipulating data in a **relational database**. Think of a relational database as a highly organized digital filing cabinet. Inside this cabinet, data is stored in **tables**.

A table is simply a collection of related data organized into **rows** and **columns**. You can think of it like a spreadsheet.

- **Columns** (or attributes) represent a specific piece of information (e.g., first_name, email, age). Each column has a defined **data type**.
- **Rows** (or records) represent a single entry or item (e.g., one specific user).

Popular database management systems (DBMS) that use SQL include **PostgreSQL**, MySQL, and SQLite. While SQL is a standard, each DBMS might have minor differences in syntax. To create a table, you use the CREATE TABLE command. You must specify the column names and their data types.

Example: Creating a users table

```
CREATE TABLE users (  
  user_id INT,      -- A whole number for the user's ID  
  first_name VARCHAR(50), -- A variable-length string for the first name (max 50 chars)  
  last_name VARCHAR(50), -- A variable-length string for the last name (max 50 chars)  
  birth_date DATE,    -- A date value (YYYY-MM-DD)  
  created_at TIMESTAMP -- A timestamp including date and time  
);
```

Common Data Types:

- INT: Whole numbers.
- VARCHAR(n): Text with a maximum length of n characters.
- TEXT: For long-form text.
- BOOLEAN: true or false values.
- DATE: Stores a date (e.g., '2025-09-28').
- TIMESTAMP: Stores a specific point in time, including date and time.

Resources

- **PostgreSQL Tutorial**: An interactive way to learn PostgreSQL.
- **PostgreSQL Data Types**: Official documentation on data types.

3.2 CRUD: The Four Essential Operations

CRUD stands for **Create**, **Read**, **Update**, and **Delete**. These are the four fundamental actions you'll perform on your data.

Create (The INSERT Statement)

The INSERT statement adds new rows of data to a table.

Example: Add a new user

```
INSERT INTO users (user_id, first_name, last_name, birth_date, created_at)  
VALUES (1, 'Alice', 'Johnson', '1995-03-14', '2025-06-08 10:00:00');
```

You can also insert multiple rows at once:

```
INSERT INTO users (user_id, first_name, last_name, birth_date, created_at) VALUES  
(2, 'Bob', 'Williams', '1998-07-22', '2025-06-08 10:05:00'),  
(3, 'Charlie', 'Brown', '2001-11-30', '2025-06-08 10:06:00');
```

Read (The SELECT Statement)

The SELECT statement retrieves data from one or more tables. This is the most frequently used SQL command.

Example: Get all data from the users table

```
SELECT * FROM users;
```

Example: Get specific columns and filter results

This query finds the name and birth date of users born after January 1st, 1998.

```
SELECT first_name, last_name, birth_date  
FROM users  
WHERE birth_date > '1998-01-01';
```

Example: Order the results

This query retrieves all users and sorts them by their first_name in alphabetical order.

```
SELECT user_id, first_name, last_name  
FROM users  
ORDER BY first_name ASC; -- ASC for ascending (A-Z), DESC for descending (Z-A)
```

Update (The UPDATE Statement)

The UPDATE statement modifies existing data in a table.

 **Always use a WHERE clause with UPDATE!** If you forget it, you will update **every single row** in the table.

Example: Change a user's last name

```
UPDATE users  
SET last_name = 'Smith'  
WHERE user_id = 1; -- This targets only the user with ID 1
```

Delete (The DELETE Statement)

The DELETE statement removes rows from a table.

 **Always use a WHERE clause with DELETE!** If you forget it, you will delete **all data** from the table.

Example: Remove a specific user

```
DELETE FROM users  
WHERE user_id = 3;
```

3.3 Constraints: Enforcing Data Integrity

Constraints are rules that you apply to columns to ensure the **quality, consistency, and reliability** of your data. They prevent invalid data from being entered into the database.

Attribute/Column Constraints

These constraints apply to a single column.

- **NOT NULL:** Ensures that a column cannot have a NULL (empty) value. This is useful for required information like an email address or a name.
- **UNIQUE:** Ensures that every value in a column is unique. Perfect for a username or email column where duplicates are not allowed.

Example:

```
CREATE TABLE accounts (  
  account_id INT,  
  username VARCHAR(50) NOT NULL UNIQUE, -- Cannot be empty and must be unique  
  password_hash VARCHAR(255) NOT NULL -- Cannot be empty  
);
```

Relational/Table Constraints

These constraints are fundamental to the "relational" model and define keys for the whole table or relationships between tables.

- **PRIMARY KEY:** The most important constraint. It uniquely identifies each row in a table. A primary key is implicitly both UNIQUE and NOT NULL. Every table should have a primary key. `user_id` is a perfect candidate.
- **FOREIGN KEY:** This is the magic that creates a **relationship between two tables**. A foreign key in one table points to the PRIMARY KEY of another table. This ensures that a value can only be entered if the same value already exists in the linked table. For example, you can't create an order for a `user_id` that doesn't exist in the users table.

Example: Creating tables with keys

-- First, create the 'users' table with a Primary Key

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  birth_date DATE,  
  created_at TIMESTAMP  
);
```

-- Next, create an 'orders' table with its own Primary Key

-- and a Foreign Key that references the 'users' table

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  order_date DATE NOT NULL,  
  total_amount DECIMAL(10, 2),  
  customer_id INT, -- This will be our Foreign Key
```

```
-- Define the Foreign Key constraint
FOREIGN KEY (customer_id) REFERENCES users(user_id)
);
```

With this setup, you cannot insert an order with a customer_id of 99 unless a user with user_id = 99 exists in the users table.

Resources

- **W3Schools SQL Constraints:** A simple tutorial on SQL constraints.

3.4 ER Diagrams: Visualizing Your Database

An **Entity-Relationship (ER) Diagram** is a flowchart that visualizes how different "entities" (like users, products, or orders) relate to each other in a database. It's a crucial first step in database design, helping you plan the structure before writing any SQL code.

A simple ER diagram has three main components:

1. **Entity:** A real-world object or concept whose data you want to store. Becomes a **table** in your database. (e.g., Student, Course).
2. **Attribute:** A property or characteristic of an entity. Becomes a **column** in your table. (e.g., firstName for the Student entity).
3. **Relationship:** Describes how two or more entities are associated. Becomes a **foreign key** link between tables.

Relationships have **cardinality**, which defines the numerical relationship between entities:

- **One-to-One (1:1):** Each record in Table A relates to exactly one record in Table B.
 - *Example:* A User has one Profile. A Profile belongs to one User.
- **One-to-Many (1:M):** Each record in Table A can relate to many records in Table B, but each record in Table B relates to only one record in Table A.
 - *Example:* One Author can write many Books. Each Book is written by one Author. This is the most common relationship type.
- **Many-to-Many (M:N):** A record in Table A can relate to many records in Table B, and a record in Table B can relate to many records in Table A.
 - *Example:* A Student can enroll in many Courses. A Course can have many Students. In a database, this is implemented using a third "junction" table (e.g., an Enrollments table connecting Students and Courses).

Resources

- **Lucidchart ER Diagram Tutorial:** A guide on how to create ER diagrams.

Your Task

Now it's time to apply what you've learned. You will design and build a small database for a **University Registration System**.

1. Design an ER Diagram:

- Create an ER diagram that includes at least the following five entities: Students, Courses, Instructors, Departments, and Enrollments.
- Your design must include one-to-one, one-to-many, and many-to-many relationships.
- For each entity, define its key attributes (e.g., `student_id`, `first_name`, `course_code`).
- Provide a brief explanation for the cardinality of each relationship (e.g., "An Instructor teaches many Courses, so it's a one-to-many relationship.").

2. Database Setup:

- Install PostgreSQL on your local machine.
- Create a new database named `university_db`.
- Write and execute the CREATE TABLE SQL scripts based on your ER diagram. For this step, do not add any PRIMARY KEY or FOREIGN KEY constraints yet.

3. Perform CRUD Operations:

- Write **five different INSERT queries** to populate your tables with sample data (e.g., add 2 departments, 5 students, 3 instructors, etc.).
- Write **five different SELECT queries** to retrieve data. Your queries should demonstrate filtering (WHERE), sorting (ORDER BY), and selecting specific columns.
- Write **five different UPDATE queries** to modify existing data (e.g., change a student's major, update a course title).
- Write **five different DELETE queries** to remove data (e.g., cancel a course, remove a student).

4. Add and Test Constraints:

- Modify your tables using the ALTER TABLE command to add constraints.
- **Attribute/Column Constraints:** Add two NOT NULL constraints and two UNIQUE constraints to appropriate columns. Explain why these are necessary for data integrity.
- **Relational/Table Constraints:** Add PRIMARY KEY constraints to all your tables. Add FOREIGN KEY constraints to establish the relationships from your ER diagram. Explain why these keys are critical.
- **Test the Constraints:** After adding constraints, try to execute queries that would violate them. For example:
 - Try to INSERT a student with a `student_id` that already exists (violates PRIMARY KEY).
 - Try to INSERT a new enrollment record for a `student_id` that does not exist in the Students table (violates FOREIGN KEY).

- Try to UPDATE a student's email to NULL if you made that column NOT NULL.
- Describe the error messages you receive from PostgreSQL and explain what they mean. This demonstrates your understanding of how constraints protect the database.